3.1 Wprowadzenie do RAG (Retrieval-Augmented Generation): Jak łączyć LLM z własną wiedzą?

Moduł 3: Budowa Aplikacji i Agenci AI

Krzysztof Rytlewski · 2026

Dlaczego same LLM nie wystarczają?

Duże modele językowe posiadają rozległą wiedzę ogólną, ale mają trzy fundamentalne ograniczenia w zastosowaniach biznesowych:

- **Data odcięcia** (*knowledge cutoff*) – model nie zna informacji nowszych niż data treningu.

- **Brak wiedzy prywatnej** – model nie zna Twoich dokumentów firmowych, bazy klientów ani wewnętrznej dokumentacji.

- **Halucynacje przy lukach w wiedzy** – gdy model nie zna odpowiedzi, zamiast przyznać się do niewiedzy, generuje wiarygodnie brzmiące, lecz nieprawdziwe odpowiedzi.

RAG (Retrieval-Augmented Generation) rozwiązuje te problemy przez **dynamiczne dostarczanie modelem językowym relevantnego kontekstu** tuż przed wygenerowaniem odpowiedzi.

RAG – analogia i zastosowania

Analogia: Egzamin z otwartą książką. Bez RAG model jest jak student uczący się wyłącznie z pamięci – szybki, ale podatny na błędy przy niestandardowych pytaniach. Z RAG model jest jak student z dostępem do biblioteki – sprawdza fakty przed odpowiedzią, a odpowiedź jest zakorzeniona w konkretnych źródłach.

RAG jest standardem przemysłowym dla:

- Systemów pytań i odpowiedzi (Q&A) nad dokumentacją,
- Chatbotów korporacyjnych z wiedzą wewnętrzną,
- Asystentów obsługi klienta,
- Systemów wyszukiwania dokumentacji technicznej.

Architektura RAG – Faza 1: Indeksowanie (offline)

Indeksowanie to jednorazowy lub periodyczny proces przygotowania wiedzy:

```
Dokumenty źródłowe
        ↓
    Chunking (podział na fragmenty)
        ↓
    Embedding (zamiana na wektory liczbowe)
        ↓
    Baza wektorowa (zapis wektorów + metadane)
```

Indeksowanie odbywa się z wyprzedzeniem i może obejmować pliki tekstowe, PDF, strony Confluence, repozytoria GitHub – dowolne źródła, które można załadować jako tekst.

Architektura RAG – Faza 2: Retrieval + Generation (na żywo)

Przy każdym zapytaniu użytkownika wykonywane są następujące kroki:

```
Pytanie użytkownika
        ↓
   Embedding pytania
        ↓
   Similarity search w bazie wektorowej
        ↓
   Top-K najtrafniejszych chunków
        ↓
   Augmented prompt = pytanie + chunki kontekstowe
        ↓
   LLM generuje odpowiedź opartą na kontekście
```

Kluczowe komponenty pipeline RAG

Komponent	Rola	Przykłady
Document loader	Wczytuje dokumenty	fs, PDF, Confluence, GitHub
Text splitter	Dzieli na chunki	RecursiveCharacterTextSplitter
Embedding model	Zamienia tekst na wektory	OpenAI `text-embedding-3-small`
Vector store	Przechowuje i przeszukuje wektory	MongoDB Atlas Vector Search
Retriever	Wyciąga relevantne chunki	similarity search, MMR
LLM	Generuje odpowiedź z kontekstem	GPT-4o, Claude 3.5 Sonnet

Implementacja RAG w Node.js – indeksowanie dokumentów

```
// scripts/index-docs.js
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,
  chunkOverlap: 200,
});
const chunks = await splitter.splitDocuments(docs);

const embeddings = new OpenAIEmbeddings({ model: 'text-embedding-3-small' });
await MongoDBAtlasVectorSearch.fromDocuments(chunks, embeddings, {
  collection,
  indexName: 'vector_index',
  textKey: 'text',
  embeddingKey: 'embedding',
});
```

Wymagane pakiety: `langchain @langchain/openai @langchain/mongodb mongodb`.

Konfiguracja indeksu wektorowego w MongoDB Atlas

W Atlas UI: Database → Atlas Search → Create Search Index → Atlas Vector Search:

```
{
  "fields": [
    {
      "numDimensions": 1536,
      "path": "embedding",
      "similarity": "cosine",
      "type": "vector"
    },
    {
      "path": "source",
      "type": "filter"
    }
  ]
}
```

Pole `filter` pozwala na późniejsze zawężanie wyników wyszukiwania do konkretnych źródeł bez wpływu na indeks wektorowy.

Chunking – sztuka dzielenia dokumentów

Jakość RAG w dużej mierze zależy od sposobu podziału dokumentów. Zbyt małe chunki tracą kontekst, zbyt duże wypełniają okno kontekstowe.

Strategia	Opis	Kiedy używać
Fixed-size	Stała liczba znaków/tokenów	Jednorodny tekst ciągły
Recursive	Podział po akapitach, zdaniach, słowach	Dokumenty ogólne (najczęściej)
Semantic	Granice zdaniowe + podobieństwo	Długie artykuły, raporty
Document-based	Podział według struktury (nagłówki Markdown)	Dokumentacja techniczna

Dobre parametry startowe: `chunkSize: 1000`, `chunkOverlap: 200`.

Metadane chunków – cytowanie źródeł

Zawsze dołączaj informacje o źródle do każdego chunku. Umożliwia to cytowanie źródeł w odpowiedzi modelu i filtrowanie wyników wyszukiwania.

```
chunks.forEach(chunk => {
  chunk.metadata.indexedAt = new Date().toISOString();
  chunk.metadata.version = '1.0';
  // chunk.metadata.source jest automatycznie
  // ustawiany przez DirectoryLoader
});
```

Metadane zapisane razem z embeddingiem pozwalają później filtrować wyniki: np. *„szukaj tylko w dokumentach z wersji 2.0"* lub *„szukaj tylko w kategorii* `api`*"*.

Ocena jakości RAG – metryki retrieval i generowania

Metryki retrieval (wyszukiwania):

- **Precision@K** – ile z K zwróconych chunków było faktycznie relevantnych.
- **Recall** – czy wśród K chunków znalazła się potrzebna informacja.
- **MRR (Mean Reciprocal Rank)** – czy najtrafniejszy chunk był wysoko na liście.

Metryki generowania:

- **Faithfulness** – czy odpowiedź wynika z kontekstu, czy model halucynuje.
- **Answer relevance** – czy odpowiedź jest relevantna do pytania.
- **Context relevance** – czy pobrane chunki były faktycznie potrzebne.

Typowe problemy i rozwiązania

Problem	Przyczyna	Rozwiązanie
Model ignoruje kontekst	Zły format prompta	Wzmocnij instrukcję systemową
Złe chunki w top-K	Niespójne embeddingi	Wypróbuj MMR zamiast similarity
Sprzeczne odpowiedzi	Sprzeczne dokumenty w bazie	Filtruj po wersji/dacie w metadanych
Wolne wyszukiwanie	Brak indeksu HNSW	Włącz indeks wektorowy w konfiguracji

Kluczowy element instrukcji systemowej wymuszający wierność kontekstowi: *„Odpowiadaj wyłącznie na podstawie dostarczonego kontekstu. Jeśli nie znasz odpowiedzi z kontekstu, powiedz: Nie znalazłem tej informacji."*

RAG jako mikroserwis Express – wzorzec integracji

Dla istniejących aplikacji rekomendowany jest wzorzec RAG jako oddzielny mikroserwis:

- Główna aplikacja komunikuje się z serwisem RAG przez HTTP.

- Serwis RAG jest niezależnie skalowalny i testowalny.

- Zmiana strategii chunking lub modelu embeddingów nie wymaga zmian w głównej aplikacji.

Endpoint RAG zwraca zarówno `answer` (treść odpowiedzi), jak i `sources` (lista plików źródłowych) – umożliwiając interfejsowi użytkownika pokazanie skąd pochodzi odpowiedź.